Information Modeling Using EXPRESS — RULES

THE PRINCIPLES

1. EXPRESS is permissive — anything that is not prohibited is allowed.
2. The rule about rules is very simple:
    - Don’t use global rules

   (except as a last resort).
3. EXPRESS does not define the time (e.g. instance creation, instance useage) at which rules are tested.

Kinds of rules

There are two kinds of rule:

Local rules
: These are defined in entities or types and apply to each and every instance.

Global rules
: These are defined outside of entities and apply to specific entity instances, or combinations of entity instances.

Constraints can also be specified by UNIQUE clauses, INVERSE attributes, subtyping with attribute redeclaration, importation (USE, REFERENCE, subtype and select pruning). Sometimes DERIVED attributes can be considered as constraints.

The NYM Principle

- If things are the same, then they should have the same name.
- If things are not the same, then they are different.
- Different things should have different names.

In general, ‘One name, one meaning, one definition’. Synonyms and homonyms in a model are a fruitful and never-ending source of confusion.

Global RULES and the Nym principle

Global rules should not change the meaning or potential for interoperability of the entity to which they are applied.

```
SCHEMA main;
  ENTITY fred;
    a : LIST OF joe;
  END_ENTITY;
...
END_SCHEMA; -- main

SCHEMA sub;
  USE FROM main (fred);
...
```

There must be at least 5 `tt "joe"`s in the list for attribute `tt "a"`.

```
RULE bad_rule FOR (fred);
LOCAL
  good : LOGICAL := TRUE;
END_LOCAL;
REPEAT i := 1 TO SIZEOF(fred);
  IF SIZEOF(fred[i].a) < 5
  THEN good := FALSE; SKIP;
  END_IF;
END_REPEAT;
WHERE
 w1: good;
END_RULE;

END_SCHEMA; -- sub
```

The constrained version of `tt "fred"` is better modeled as:

```
SCHEMA sub;
  REFERENCE FROM main (fred);

ENTITY sub_fred
  SUBTYPE OF (fred);
WHERE
 w1: SIZEOF(SELF\fred.a) >= 5;
END_ENTITY;
```

Or, perhaps even better as:

```
ENTITY sub_fred
  SUBTYPE OF (fred);
  SELF\fred.a : LIST [5:?] OF joe;
END_ENTITY;
```

Local (WHERE) Rules

Are defined within the definition of a type or entity and apply to each and every instance of the type or entity.

```
WHERE
  label_1: logical_expression_1 ;
  label_2: logical_expression_2 ;
  ....
END_
```

- An instance is non-conforming if any logical expression evaluates to FALSE.
- An instance is conforming if all the logical expressions evaluate to TRUE.
- An instance is considered to be not non-conforming if some or all the logical expressions evaluate to UNKNOWN and the remainder evaluate to TRUE.

‘Logical’ Rule

This domain rule may evaluate to FALSE, UNKNOWN or TRUE.

```
ENTITY vector;
  x, y : REAL;
  z    : OPTIONAL REAL;
WHERE
  w1: x**2 + y**2 + z**2 = 1.0;
END_ENTITY;
```

‘Boolean’ Rule

This domain rule will only evaluate to FALSE or TRUE.

```
ENTITY vector;
  x, y : REAL;
  z    : OPTIONAL REAL;
WHERE
  w1: x**2 + y**2 + NVL(z, 0.0)**2 = 1.0;
END_ENTITY;
```

NOTE If `tt "x"` or `tt "y"` does not have a value in a particular instance of `tt "vector"`, then the instance is non-conforming by definition.

‘Function’ Rule

A rule can be described using a logical (or boolean) function.

```
ENTITY vector;
  x, y : REAL;
  z    : OPTIONAL REAL;
WHERE
  w1: unit_vector(SELF);
END_ENTITY;

FUNCTION unit_vector(v:vector):BOOLEAN;
  RETURN(v.x**2 + v.y**2 +
         NVL(v.z, 0.0)**2 = 1.0);
END_FUNCTION;
```

Functions are of most use when it is difficult to express the constraint as a single logical expression. They are also beneficial when the same constraint applies to different kinds of things.

```
ENTITY vector;
  x, y : REAL;
  z    : OPTIONAL REAL;
WHERE
  w1: unit_vector(x,y,z);
END_ENTITY;

FUNCTION unit_vector(u,v,w:REAL):LOGICAL;
  IF (NOT EXISTS(w)) THEN
    IF (NOT EXISTS(v)) THEN
      RETURN(u**2 = 1.0);
    END_IF;
    RETURN(u**2 + v**2 = 1.0);
  END_IF;
  RETURN(u**2 + v**2 + w**2 = 1.0);
END_FUNCTION;
```

UNIQUE

```
ENTITY circle;
  centre : point;
  radius : positive_number;
END_ENTITY;
```

There can be any number of circles in the object base with identical centres and/or radii.

```
ENTITY circle;
  centre : point;
  radius : positive_number;
UNIQUE
  not_concentric : centre;
END_ENTITY;
```

There can be any number of circles in the object base with identical radii but none with identical centres. (No circles are concentric but some may have the same size.)

```
ENTITY circle;
  centre : point;
  radius : positive_number;
UNIQUE
  different_sizes : radius;
END_ENTITY;
```

There can be any number of circles in the object base with identical centres but none with identical radii. (No circles have the same size but some may be concentric.)

```
ENTITY circle;
  centre : point;
  radius : positive_number;
UNIQUE
  not_concentric : centre
  different_sizes : radius;
END_ENTITY;
```

There can be no circles in the object base with identical centres and no circles with identical radii. (Every circle is a different size and differently located.)

```
ENTITY circle;
  centre : point;
  radius : positive_number;
UNIQUE
  all_different : centre, radius;
END_ENTITY;
```

There can be no circles in the object base with the identical combination of centre and radius. (No circles represent the same ‘point set’.)

Instance and Value

In EXPRESS comparisons for uniqueness are performed on the ‘object-id’ for entity instances, and on values for ‘anonymous’ types (e.g. REAL). Thus,

```
TYPE pair = SET [2:2] OF point;
END_TYPE;
```

requires that `pair[1] :<>: pair[2]` is TRUE, but `pair[1] = pair[2]` may be TRUE or FALSE.

For value uniqueness, do something like:

```
TYPE vpair = SET [2:2] OF point;
WHERE
 vun: VALUE_UNIQUE(SELF);
END_TYPE;
```

which requires `vpair[1] = vpair[2]` to be FALSE.

Joint value uniqueness

```
ENTITY e;
 a1 : a;
 a2 : b;
 a3 : c
UNIQUE
  ju : a1, a2;
END_ENTITY;
```

The values of the attributes `tt "a1"` and `tt "a2"` are constrained to be jointly *instance unique*.

If they are further required to be jointly *value unique*, use a global rule of the following kind to specify this additional constraint.

```
RULE vu FOR (e);
  ENTITY temp;
    a1 : a;
    a2 : b;
  END_ENTITY;
LOCAL
  s : SET OF temp := [];
END_LOCAL;
REPEAT i := 1 TO SIZEOF(e);
  s := s + temp(e[i].a1, e[i].a2);
END_REPEAT;
WHERE
  jvu: VALUE_UNIQUE(s);
END_RULE;
```

Note the use of an ENTITY definition local to the rule, and the use of the entity constructor for

instances of this entity type.

GLOBAL RULES

Are defined outside entities and only apply to entities. Every instance of the specified entity(s) is examined. The entity instances are conforming the WHERE rules all evaluate to TRUE.

```
RULE rname FOR (ent1, ent2, ...);
  body of rule (code)
WHERE
  label_1: logical_expression_1 ;
   ...
END_RULE;
```

All instances of entities of the given type(s) are examined during rule execution (combinatorial explosion?).

Global Rule Usage

Use a global rule when:

1. A combination of different entity types must be constrained; or
2. A constraint only applies to some, but not all, instances of a particular entity type; or
3. The number of instances is to be constrained.

Person Example

```
ENTITY person;
  name   : STRING;
  ss_no  : INTEGER;
  sex    : gender;
  spouse : OPTIONAL person;
UNIQUE
  un1: ss_no;
END_ENTITY;
```

Person Example

```
ENTITY person;
  name   : STRING;
  ss_no  : INTEGER;
  gender : sex;
  spouse : OPTIONAL person;
```

```
UNIQUE
  un1: ss_no;
WHERE
  w1: (EXISTS(spouse) AND
       gender <> spouse.gender)
      XOR (NOT EXISTS(spouse));
END_ENTITY;
```

Person Example

```
ENTITY person;
  name  : STRING;
  ss_no : INTEGER;
UNIQUE
  un1: ss_no;
END_ENTITY;

ENTITY male
  SUBTYPE OF (person);
  wife : OPTIONAL female;
END_ENTITY;

ENTITY female
  SUBTYPE OF (person);
  husband : OPTIONAL male;
END_ENTITY;
```

Person Example

```
ENTITY person
  SUPERTYPE OF (ONEOF(male,female));
  name  : STRING;
  ss_no : INTEGER;
UNIQUE
  un1: ss_no;
END_ENTITY;

ENTITY male
  SUBTYPE OF (person);
  wife : OPTIONAL female;
END_ENTITY;

ENTITY female
  SUBTYPE OF (person);
  husband : OPTIONAL male;
```

```
END_ENTITY;
```

Example — Married rule

```
RULE married FOR (male, female);
  LOCAL
    ok1, ok2 : BOOLEAN := TRUE;
  END_LOCAL;
  IF (EXISTS(male.wife) AND
      male :<>: male.wife.husband) THEN
    ok1 := FALSE;
  END_IF;
  IF (EXISTS(female.husband) AND
      female :<>: female.husband.wife) THEN
    ok2 := FALSE;
  END_IF;
WHERE
  r1: ok1;
  r2: ok2;
END_RULE;
```

Example — Married entity

```
ENTITY male SUBTYPE OF (person);
END_ENTITY;

ENTITY female SUBTYPE OF (person);
END_ENTITY;

ENTITY married;
  husband : male;
  wife    : female;
UNIQUE
  no_bigamy: husband;
  no_polyandry: wife;
END_ENTITY;
```

Limit instances

```
CONSTANT
max_scj : INTEGER := 9;
END_CONSTANT;

ENTITY scj SUBTYPE OF (person);
```

```
END_ENTITY;

RULE max_no FOR (scj);
WHERE
  r1: SIZEOF(scj) <= max_scj;
END_RULE;
```

This rule says that there shall be no more than `max_scj tt "scj"`s (Supreme Court Justices).

The following RULE states that there shall be one and only one point at the origin in the object-base.

```
RULE unique_origin FOR (point);
LOCAL
  origin : SET OF point;
END_LOCAL;
  origin := QUERY(temp <* point |
                  (temp.x = 0.0) AND
                  (temp.y = 0.0));
WHERE
  r1: SIZEOF(origin) = 1;
END_RULE;
```

Recursion — Entity

This `tt "node"` entity is ‘type recursive’ and may be ‘instance recursive’

```
ENTITY node;
  local_data : data;
  children : LIST OF UNIQUE node;
END_ENTITY
```

This `tt "node"` entity is ‘type recursive’ and not ‘self instance recursive’ but may be ‘globally instance recursive’.

```
ENTITY node;
  local_data : data;
  children : LIST OF UNIQUE node;
WHERE
 all_unique : NOT (SELF IN SELF.children);
END_ENTITY;
```

Recursion — Function

```
FUNCTION NodeSet(input: node): SET OF node;
```

```
LOCAL
  result : SET OF node := [];
END_LOCAL;
REPEAT i := 1 TO SIZEOF(input.children);
  result := result + NodeSet(input.children[i]);
END_REPEAT;
RETURN(result + input);
END_FUNCTION;
```

```
FUNCTION NodeBag(input: node): BAG OF node;
LOCAL
  result : BAG OF node := [];
END_LOCAL;
REPEAT i := 1 TO SIZEOF(input.children);
  result := result + NodeBag(input.children[i]);
END_REPEAT;
RETURN(result + input);
END_FUNCTION;
```

RULE with recursive functions

A tree of nodes must be acyclic. That is, a given node instance must only appear once in the tree.

```
RULE acyclic_tree FOR (node);
LOCAL
  result : LOGICAL;
END_LOCAL;
REPEAT i := 1 TO SIZEOF(node);
  result := SIZEOF(NodeSet(node[i])) =
            SIZEOF(NodeBag(node[i]));
  IF (result = FALSE)
  THEN SKIP;
  END_IF;
END_REPEAT;
WHERE
  acyclic: result;
END_RULE;
```

Or

```
RULE acyclic_tree FOR (node);
WHERE
  acyclic: SIZEOF(QUERY(t <* node |
                   SIZEOF(NodeSet(t)) <>
                   SIZEOF(NodeBag(t)))
                  ) = 0;
```

```
END_RULE;
```

More Recursion

This can be used to describe a relationship between two `tt "obj"` (Part 41, Annex D).

```
ENTITY relationship;
  description : STRING;
  parent      : obj;
  child       : obj;
END_ENTITY;
```

In turn, the `tt "obj"` that is a child in one of these may be the parent in another `tt "relationship"`, and so on. Often it is required that a string of `tt "relationship"` be acyclic. More simply, a child cannot be its own ancestor, or equivalently a parent cannot be its own descendent.

Use a function in a WHERE rule as:

```
WHERE
w1: acyclic(SELF,[SELF.parent],'...');
```

Convert an AGGREGATE to a SET.

```
FUNCTION Agg2Set(agg: AGGREGATE OF GENERIC:a):
                 SET OF GENERIC:a;
LOCAL
  result : SET OF GENERIC:a := [];
END_LOCAL;
REPEAT i := LOINDEX(agg) TO HIINDEX(agg);
  result := result + agg[i];
END_REPEAT;
RETURN(result);
END_FUNCTION;
```

Does the following (Part 41 p 156) work?

```
FUNCTION acyclic(rel: relationship;
                 relatives: SET [1:?] OF obj;
                 subtyp: STRING): LOGICAL;
LOCAL
  x     : SET [1:?] OF relationship;
  close : SET [1:?] OF obj;
END_LOCAL;
REPEAT i := 1 TO HIINDEX(relatives);
  IF rel.parent :=: relatives[i]
```

```
  THEN RETURN(FALSE); END_IF;
END_REPEAT;
x := Agg2Set(USEDIN(rel.parent, subtyp));
close := relatives + rel.parent;
REPEAT i := 1 TO SIZEOF(x);
  IF NOT acyclic(x[i],close,subtyp)
    THEN RETURN(FALSE); END_IF;
END_REPEAT;
RETURN(TRUE);
END_FUNCTION;
```

TYPEOF function

`TYPEOF(V: GENERIC): SET OF STRING;` returns the set of uppercase strings holding the fully qualified names of the types of which the value (instance) `tt "V"` could be a value of. That is, the result is the set of potential uses of `tt "V"`, not the actual usage.

```
SCHEMA s;

TYPE mylist = LIST OF REAL; END_TYPE;
...
LOCAL lst : mylist; END_LOCAL;

TYPEOF(lst) = ['S.MYLIST', 'LIST']; -- TRUE
```

Note that given a subtype instance, the returned set will include the subtype and all its supertypes, but it excludes subtypes lower in the tree.

SIZEOF function

`SIZEOF(agg)` returns the number of element instances in the (aggregate) instance `agg`.

Usually used for controlling an iteration or for comparing the actual sizes of two aggregates.

```
ENTITY PoorExamMarks;
  course   : STRING;
  students : LIST OF UNIQUE person;
  marks    : LIST OF INTEGER;
WHERE
  matched_lists : SIZEOF(students) =
                  SIZEOF(marks);
END_ENTITY;
```

This has been used as an attempt to specify that there is a one-to-one correlation between the elements in the two lists.

Correlated aggregates

```
ENTITY BetterExamMarks;
  course : STRING;
  results : LIST OF StudentMark;
END_ENTITY;

ENTITY StudentMark;
  student : person;
  mark    : INTEGER;
END_ENTITY;
```

But what about student uniqueness in `tt "BetterExamMarks"`?

```
ENTITY BestExamMarks;
  course : STRING;
  results : LIST OF StudentMark;
WHERE
  wr1: UniqueStudents(results);
END_ENTITY;
```

UniqueStudents

```
FUNCTION UniqueStudents
         (input: AGGREGATE OF StudentMark):
         LOGICAL;
LOCAL
  aBag : BAG OF person := [];
END_LOCAL;
REPEAT i := 1 TO SIZEOF(input);
  aBag := aBag + input[i].student;
END_REPEAT;
RETURN (SIZEOF(aBag) =
        SIZEOF(Agg2Set(aBag)));
END_FUNCTION;
```

QUERY function

`QUERY(v <* InAgg | Lexp(v)): OutAgg`
applies the logical expression `tt "Lexp(v)"` to each element of the aggregate `tt "InAgg"`. Each element for which `tt "Lexp"` is TRUE is added to the returned aggregate `tt "OutAgg"`, which is of the same type as `tt "InAgg"`. It is equivalent to the following pseudo-EXPRESS.

```
FUNCTION query(input: AGGREGATE OF GENERIC:GEN;
               LEXP):
              AGGREGATE OF GENERIC:GEN;
LOCAL
  result : AGGREGATE OF GENERIC:GEN := [];
END_LOCAL;
REPEAT i := LOINDEX(input) TO HIINDEX(input);
  IF Lexp(input[i]) = TRUE
  THEN  result := result + input[i];
  END_IF;
END_REPEAT;
RETURN(result);
END_FUNCTION;
```

Example

A school party must have at least one adult for every 10 children and shall not be larger than 50 in total.

```
ENTITY SchoolParty;
  adults, children : SET OF person;
WHERE
  w1: 10*SIZEOF(adults) >= SIZEOF(children);
  w2: SIZEOF(adults) + SIZEOF(children) <= 50;
END_ENTITY;
```

Or, reformulating the entity and using the QUERY function:

```
ENTITY SchoolParty;
  group : SET [2:50] OF person;
WHERE
w1: 10*SIZEOF(QUERY(p <* group | p.age >= 21))
    >=
    SIZEOF(QUERY(p <* group | p.age <= 18));
END_ENTITY;
```

QUERY and SIZEOF

`QUERY` and `SIZEOF` functions are often combined.

```
SIZEOF(QUERY(p <* e | Bad(p)=TRUE)) = 0;

SIZEOF(QUERY(p <* e | MaxOneBad(p)=TRUE)) <= 1;
```

```
SIZEOF(QUERY(p <* e | AtLeastOne(p)=TRUE)) >0;

{2 <=
  SIZEOF(QUERY(p <* e | Two2Five(p)=TRUE))
<= 5};

SIZEOF(QUERY(p <* e | AllGood(p)=TRUE))
= SIZEOF(e);
```

USEDIN function

`tt "USEDIN(T:GENERIC; R:STRING): BAG OF GENERIC;"` returns the BAG of entity instances that uses instance `tt "T"` in role `tt "R"`.

If `tt "T"` plays no roles and/or role `tt "R"` is not found, the returned BAG is empty.

If `tt "R"` is an empty string, every usage of instance `tt "T"` is reported.

Note that the `tt "USEDIN"` function examines instances in an object-base. That is, it looks at actual data rather than the potential kinds (types) of data.

USEDIN example

```
ENTITY PoorEnt;
  attr : PoorColour;
END_ENTITY;

ENTITY PoorColour;
  hue        : fraction;
  saturation : fraction;
  intensity  : fraction;
WHERE
  wr1: SIZEOF(QUERY(x <*
              USEDIN(SELF, 'POORENT.ATTR') |
       (x.attr.intensity > 0.5))) = 0;
END_ENTITY;
```

Says that when an instance of `tt "PoorColour"` is used as the `tt "attr"` of the entity `tt "PoorEnt"`, then its value for `tt "intensity"` shall be not more than half.

This model is better written as:

```
ENTITY Ent;
  attr : Colour;
WHERE
  wr1: attr.intensity <= 0.5;
END_ENTITY;
```

```
ENTITY Colour;
  hue        : fraction;
  saturation : fraction;
  intensity  : fraction;
END_ENTITY;
```

Or, it could be rewritten using an inverse.

```
ENTITY Ent;
  attr : Colour;
END_ENTITY;

ENTITY Colour;
  hue        : fraction;
  saturation : fraction;
  intensity  : fraction;
INVERSE
  low : BAG OF Ent FOR attr;
WHERE
  w1: (SIZEOF(low) > 0 AND
       intensity <= 0.5) XOR
      (SIZEOF(low) = 0);
END_ENTITY;
```

USEDIN example

```
RULE SecondClass FOR (ent);
WHERE
  wr1: SIZEOF(QUERY(e <* ent |
              NOT (SIZEOF(USEDIN(e,'')) >= 1 )))
       = 0;
END_RULE;
```

states that `tt "ent"` shall not be independently instantiated.

- `USEDIN(e,'')` gives entities that reference instance `tt "e"` of entity type `tt "ent"`
- `SIZEOF(USEDIN(e,'')) >= 1` gives number of entities referencing `tt "e"`
- `NOT (SIZEOF⋯)` gives an `tt "e"` that is not referenced
- and there should be none of these.

The semantics of this rule are exactly the same as the EXPRESS REFERENCE construct.

```
SCHEMA good;        SCHEMA ap;
REFERENCE FROM sub    ENTITY ent;
```

```
          (ent);        ...
  ...                   ...
END_SCHEMA;           END_ENTITY;
SCHEMA sub;
ENTITY ent;           RULE SecondClass FOR
   ...                                (ent);
END_ENTITY;             ...
...
END_SCHEMA;           END_SCHEMA;
```

ROLESOF function

`ROLESOF(V:GENERIC): SET OF STRING;` returns the set of roles that the instance `tt "V"` plays in the object base.

```
SCHEMA uk;
ENTITY judge;
  office_holder : person;
  court         : STRING;
END_ENTITY;

ENTITY criminal;
  prisoner : person;
  gaol     : address;
  crime    : ...
END_ENTITY;
```

In the UK schema, a person who is a judge shall not be a prisoner in gaol.

```
RULE NoCriminalJudge FOR (person);
WHERE
wr1: SIZEOF(QUERY(p <* person |
      'UK.CRIMINAL.PRISONER' IN ROLESOF(p)
      AND
      'UK.JUDGE.OFFICE_HOLDER' IN ROLESOF(p))
     ) = 0;
END_RULE;
```

Required Optional Attributes

At least one of the optional attributes must have a value:

```
ENTITY ent;
  attr1 : OPTIONAL ...;
  attr2 : OPTIONAL ...;
```

```
WHERE
 at_least_one : EXISTS(attr1) OR
                EXISTS(attr2);
END_ENTITY;
```

One and only one of the optional attributes must have a value:

```
ENTITY ent;
  attr1 : OPTIONAL ...;
  attr2 : OPTIONAL ...;
WHERE
 only_one : EXISTS(attr1) XOR
            EXISTS(attr2);
END_ENTITY;
```

Attribute Redeclaration

```
ENTITY t;
  a : LIST OF d;
  b : NUMBER;
END_ENTITY;

ENTITY sub
  SUBTYPE OF (t);
  SELF\t.a : LIST [1:4] OF UNIQUE e;
  SELF\t.b : INTEGER;
END_ENTITY;

ENTITY e SUBTYPE OF d;
...
END_ENTITY;
```

Instead of:

```
ENTITY sub
  SUBTYPE OF (t);
WHERE
  w1: 'INTEGER' IN TYPEOF(SELF\t.b);
  w2: {1 <= SIZEOF(SELF\t.a) <= 4};
  w3: SIZEOF(SELF\t.a) =
      SIZEOF(Agg2Set(SELF\t.a));
--  w4: subtyping of list elements
END_ENTITY;
```

SUMMARY

- An EXPRESS information model is permissive (i.e. what is not explicitly prohibited is permissable).
- Minimise constraints (enhances re-useability).
- Add all necessary constraints — a model is as much about the limitations of objects as about the objects themselves.
- Specify constraints by the following ordered preferences:

  1. Model structure
  2. Local constraints
  3. Global rules